RETAILMART V3 · SQL

Window Functions Part 1 -- Ranking & Bucketing

— Siraj Sir

📱 WhatsApp Announcement

Window Functions Part 1 (Ranking & Bucketing)

Have you ever tried to find 'the top 3 products in EACH category' using GROUP BY? GROUP BY collapses rows -- you lose the product details. Enter WINDOW FUNCTIONS -- they let you rank, sum, and average across groups WHILE KEEPING EVERY ROW INTACT.

Part 1: OVER clause & PARTITION BY -- aggregate without collapsing
Part 2: ROW_NUMBER, RANK, DENSE_RANK -- three ranking strategies
Part 3: NTILE -- bucketing rows into quartiles and percentiles
Part 4: Top N per Category -- the #1 SQL interview question

Window functions appear in ~50% of data analyst interviews at top tech companies. The 'Top N per Group' pattern alone accounts for 30%+ of all SQL interview questions.

Prerequisites: GROUP BY (Aggregates & Grouping) and CTEs (CTEs). Window functions build on both.

RECAP

earlier sessions Recap

Last two sessions covered review and theory:

- Database Concepts -- Database Concepts: Normalization (1NF/2NF/3NF), ACID properties, transactions
- Review & Practice -- Review & Practice: 8 mixed problems spanning earlier sessions (joins, CTEs, subqueries, aggregation)

Today we learn a fundamentally new concept: window functions. They give you GROUP BY's aggregation power WITHOUT collapsing rows. Everything connects -- CTEs from CTEs will wrap today's window functions for filtering.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: OVER Clause & PARTITION BY	30 min	Window functions vs GROUP BY, PARTITION BY for per-group windows, Percentage of total, comparison to group average
Part 2: Ranking Functions	30 min	ROW_NUMBER -- unique sequence, deduplication, RANK -- ties with gaps, DENSE_RANK -- ties without gaps, Nth highest
Part 3: NTILE -- Bucketing	20 min	NTILE(N) for quartile/decile/percentile bucketing, Customer spend quartiles, RFM prep (Cohort & RFM)
Part 4: Top N per Category	40 min	CTE + ranking + WHERE filter pattern, Choosing ROW_NUMBER vs DENSE_RANK for Top N, Multi-column PARTITION BY for nested groupings

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Basic Queries
- ✓ Data Types
- ✓ Constraints
- ✓ Filtering
- ✓ Scalar Functions
- ✓ Conditional Logic
- ✓ Aggregates
- ✓ Joins Part 1
- ✓ Joins Part 2
- ✓ Self, Cross & Sets
- ✓ Subqueries Part 1
- ✓ CTEs & Correlated
- ✓ DB Concepts
- ✓ Review & Practice
- 1 Window Functions P1 ← HERE

STORY

The Store Manager's Monday Dashboard

Every Monday, the RetailMart Mumbai store manager opens her dashboard. She has three questions -- and GROUP BY alone cannot answer any of them cleanly.

Question 1: 'For every employee, show their salary AND their store's average salary -- on the SAME row.' GROUP BY collapses to one row per store. The employees vanish.

Question 2: 'Number each customer's orders chronologically -- their 1st order, 2nd order, 3rd. And rank products by price where ties share the same rank.' She needs ranking functions that handle ties differently.

Question 3: 'Show me the top 3 products by revenue in EACH category. Not 3 products total -- 3 per category.' LIMIT 3 gives 3 rows across the whole table. One category dominates, the rest get nothing.

Each question maps to today's four parts. By the end, you will solve all of them with window functions.

REVEAL

What Makes Window Functions Special

The secret is in the name: window functions work through a 'window' of rows related to the current row.

Three key differences from GROUP BY:

- ROWS ARE NEVER COLLAPSED -- every input row produces one output row
- You can use MULTIPLE window functions in the same SELECT, each with its own PARTITION BY
- Window functions execute AFTER WHERE, GROUP BY, and HAVING -- they see the filtered, grouped result

SQL Execution Order (critical): FROM > WHERE > GROUP BY > HAVING > SELECT (window functions here!) > ORDER BY > LIMIT. Since window functions run at SELECT, you CANNOT reference them in WHERE -- this is why CTEs are needed for filtering.

STORY

Two Ways to Compare an Employee to Their Store

HR asks: 'List every employee, their salary, and the total salary bill of their store on the same row.'

GROUP BY approach: SELECT store_id, SUM(salary) FROM stores.employees GROUP BY store_id. Returns one row per store. Individual employees are gone. Fails the requirement.

Window approach: SELECT first_name, salary, SUM(salary) OVER (PARTITION BY store_id) AS store_total FROM stores.employees. All rows returned. Each row tagged with its store's total. Nothing collapsed.

That is all OVER() does: run an aggregate WITHOUT collapsing rows. PARTITION BY store_id says 'compute the aggregate separately for each store.' Omit PARTITION BY and the entire table becomes one giant window.

DEFINITION

Window Functions & OVER Clause

Technical:

A window function performs calculations across a set of rows related to the current row, defined by the OVER() clause. Unlike GROUP BY (which collapses rows), window functions retain every individual row while computing aggregates across the 'window' of related data. They execute after WHERE, GROUP BY, and HAVING in the SQL pipeline.

Simple:

A window function adds one extra column to each row -- a column whose value is computed from a group of related rows -- and leaves every row in place. Run it against 3,000 employees and you still get 3,000 rows back. Each row just gains an extra piece of context.

OVER() anatomy:

OVER() = entire table is one window. OVER(PARTITION BY store_id) = separate window per store. OVER(PARTITION BY store_id ORDER BY salary DESC) = per-store window, rows ordered by salary. PARTITION BY is optional. ORDER BY is required for ranking functions.

GROUP BY vs Window Function -- Visual

```
RAW DATA (3 rows):
Row 1: Store A | Alice | 50000
Row 2: Store A | Bob   | 60000
Row 3: Store B | Carol | 70000

GROUP BY store (2 rows -- individuals LOST):
Row 1: Store A | TOTAL: 110000
Row 2: Store B | TOTAL: 70000

SUM(salary) OVER(PARTITION BY store) (3 rows KEPT!):
Row 1: Store A | Alice | 50000 | Store_Total: 110000
Row 2: Store A | Bob   | 60000 | Store_Total: 110000
Row 3: Store B | Carol | 70000 | Store_Total: 70000
```

OVER Clause -- Syntax Variations

```sql
-- 1. Empty OVER(): entire table is one window
SELECT first_name, salary,
       SUM(salary) OVER () AS company_total
FROM stores.employees;

-- 2. PARTITION BY: separate window per group
SELECT first_name, store_id, salary,
       SUM(salary) OVER (PARTITION BY store_id)
         AS store_total
FROM stores.employees;

-- 3. Multiple windows in one SELECT
SELECT first_name, salary,
       AVG(salary) OVER (PARTITION BY store_id)
         AS store_avg,
       AVG(salary) OVER () AS company_avg
FROM stores.employees;
```

Easy: Employee's Share of Company Payroll

EASY

SCENARIO: The CEO is preparing a town-hall slide titled 'Our People -- Our Payroll' that lists every employee with their salary, the company's total salary bill, and the percentage each person represents. GROUP BY would return just one row -- the total -- killing the slide.

TASK: Return all employees with 4 columns: first_name, salary, company_total, and pct_of_total. Use window functions, no GROUP BY, no subqueries.

HINT: SUM(salary) OVER() with no PARTITION BY treats the entire table as one window. Every row gets the same grand total. Then divide salary by that total for the percentage.

Answer

✅ ANSWER

```
SELECT first_name, salary,
       SUM(salary) OVER () AS company_total,
       ROUND(salary * 100.0
             / SUM(salary) OVER (), 2)
         AS pct_of_total
FROM stores.employees;
```

Medium: Employee vs Store Average

MEDIUM

SCENARIO: The HR team wants to see every employee's salary alongside the average salary at their store. Employees below their store average will be flagged for the raise committee. They need every row visible -- not a summary table.

TASK: For each employee, show first_name, store_id, salary, avg_store_salary, and the difference (salary minus store average). Positive = above average, negative = below.

HINT: PARTITION BY store_id creates separate windows per store. Each employee sees their OWN store's average, not the company average.

Answer

✅ ANSWER

```
SELECT first_name, store_id, salary,
       ROUND(AVG(salary)
         OVER (PARTITION BY store_id))
         AS avg_store_salary,
       salary - ROUND(AVG(salary)
         OVER (PARTITION BY store_id))
         AS difference
FROM stores.employees
ORDER BY store_id, salary DESC;
```

Hard: Salary Band Classification

HARD

SCENARIO: It is appraisal season at RetailMart. The HR Business Partner needs every employee labeled as Above, At, or Below their store's average salary. The old correlated-subquery version took 40 seconds. She needs it in under a second.

TASK: Return all employees with salary, store average, signed delta, and a status label (Above / At / Below). Use CASE with the window function result. Multiple window functions can share the same PARTITION BY.

HINT: PostgreSQL computes the partition once and reuses it across multiple window functions in the same SELECT. Three OVER(PARTITION BY store_id) calls do NOT mean three separate computations.

Answer

✅ ANSWER

```
SELECT e.first_name, e.role,
       e.store_id, e.salary,
       ROUND(AVG(e.salary)
         OVER (PARTITION BY e.store_id))
         AS store_avg,
       e.salary - ROUND(AVG(e.salary)
         OVER (PARTITION BY e.store_id))
         AS vs_avg,
       CASE
         WHEN e.salary > AVG(e.salary)
           OVER (PARTITION BY e.store_id)
           THEN 'Above'
         WHEN e.salary = AVG(e.salary)
           OVER (PARTITION BY e.store_id)
           THEN 'At'
         ELSE 'Below'
       END AS status
FROM stores.employees e
ORDER BY e.store_id, e.salary DESC;
```

Crazy: Executive Revenue Dashboard -- Three Windows

CRAZY

SCENARIO: The CFO wants a single dashboard row per store showing four numbers: the store's revenue, its city's average revenue, the company average revenue, and the store's percentage of company-wide revenue. One query. One table. No four-tab spreadsheet.

TASK: Aggregate orders to per-store revenue in a CTE. Then use three different OVER() clauses: city average (PARTITION BY city), company average (empty OVER), and percentage of total (store / SUM OVER). Sort by revenue descending.

HINT: Three different PARTITION BY clauses in one SELECT. Each creates a different context layer. CTE pre-aggregates the raw orders so the window functions operate on 200 store rows instead of 150,000 order rows.

Answer

✅ ANSWER

```
WITH store_rev AS (
    SELECT store_id,
           SUM(net_total) AS revenue
    FROM sales.orders
    GROUP BY store_id
)
SELECT s.store_name, s.city,
       sr.revenue,
       ROUND(AVG(sr.revenue)
         OVER (PARTITION BY s.city))
         AS city_avg,
       ROUND(AVG(sr.revenue)
         OVER ()) AS company_avg,
       ROUND(sr.revenue * 100.0
         / SUM(sr.revenue) OVER (), 2)
         AS pct_of_company
FROM stores.stores s
JOIN store_rev sr
  ON s.store_id = sr.store_id
ORDER BY sr.revenue DESC;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

OVER() turns aggregates into window functions. Empty OVER() = entire table window. OVER(PARTITION BY X) = per-group window. Rows are NEVER collapsed. You can use MULTIPLE window functions in the same SELECT with different PARTITION BY clauses. Window functions execute AFTER WHERE -- you cannot reference them in WHERE directly.

PRO TIP

PRO TIP

When you see the same PARTITION BY in multiple window functions, PostgreSQL computes the partition once and reuses it. Using AVG() OVER(PARTITION BY store_id) three times is NOT three separate computations -- the optimizer is smart. Do not fear multiple window functions in one SELECT.

COMMON MISTAKE

Common Mistakes with OVER Clause

Mistake 1: Window function in WHERE

WHERE salary > AVG(salary) OVER (PARTITION BY store_id) -- ERROR! Window functions execute after WHERE.

Fix:

Wrap in a CTE: WITH ranked AS (SELECT *, AVG(salary) OVER(PARTITION BY store_id) AS avg FROM employees) SELECT * FROM ranked WHERE salary > avg.

Mistake 2: Adding ORDER BY inside OVER for aggregates

SUM(salary) OVER(PARTITION BY store_id ORDER BY salary) creates a RUNNING TOTAL, not a group total! ORDER BY inside OVER changes aggregate behavior.

Fix:

For group totals, omit ORDER BY: SUM(salary) OVER(PARTITION BY store_id). Running totals are covered in Window Functions Part 2.

STORY

Three Ways to Handle a Tie

Inside a brand, four products are sorted by price descending: Product A at 1,200. Product B at 1,200. Product C at 900. Product D at 700. Two products tie at 1,200. What rank does Product C get?

ROW_NUMBER: 1, 2, 3, 4. Forces unique numbers. One tied product arbitrarily gets 1, the other 2. Product C gets 3. Use when you need a unique sequence -- deduplication, pagination, finding firsts.

RANK: 1, 1, 3, 4. Tied products share rank 1. They consumed positions 1 AND 2, so the next rank is 3. Like Olympic medals -- two golds, skip silver, next is bronze. Use for competitive leaderboards.

DENSE_RANK: 1, 1, 2, 3. Tied products share rank 1 as one tier. Next distinct price is tier 2. No gaps ever. Use for 'Nth highest salary' problems where you want the Nth unique value.

DEFINITION

ROW_NUMBER, RANK, DENSE_RANK

ROW_NUMBER():

Assigns a unique sequential integer to each row within a partition, starting at 1. NEVER assigns the same number twice. Ties are broken arbitrarily unless you add a tiebreaker column to ORDER BY. Use for: sequencing events, deduplication (keep rn = 1), pagination, finding firsts/lasts.

RANK():

Tied values share the same rank number, then SKIP. If 3 rows tie at rank 2, the next row gets rank 5 (positions 2, 3, 4 were consumed). Use for: competitive leaderboards where ties mean equal placement.

DENSE_RANK():

Tied values share the same rank, NO skip. If 3 rows tie at rank 2, the next row gets rank 3. Counts distinct values (tiers), not positions. Use for: 'Nth highest salary' problems where you need the Nth unique value.

The Big Comparison -- Same Data, Three Functions

```
Score | ROW_NUMBER | RANK | DENSE_RANK
 100  |     1      |   1  |     1
  95  |     2      |   2  |     2
  95  |     3      |   2  |     2   -- TIE!
  80  |     4      |   4  |     3
  70  |     5      |   5  |     4

-- ROW_NUMBER: 1,2,3,4,5 -- unique, no ties
-- RANK:       1,2,2,4,5 -- ties + gaps
-- DENSE_RANK: 1,2,2,3,4 -- ties, no gaps
```

Ranking Functions -- Syntax

```sql
-- All three use the same OVER() syntax
SELECT first_name, salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
    RANK()       OVER (ORDER BY salary DESC) AS rnk,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM stores.employees;

-- With PARTITION BY (ranking within groups)
SELECT first_name, store_id, salary,
    ROW_NUMBER() OVER (
        PARTITION BY store_id
        ORDER BY salary DESC
    ) AS store_rank
FROM stores.employees;

-- ORDER BY is REQUIRED for ranking functions
-- Always add a tiebreaker for deterministic results:
-- ORDER BY salary DESC, employee_id ASC
```

Easy: Global Order Ranking

EASY

SCENARIO: The ops lead is building a 'Record Orders Wall' for the quarterly all-hands. He wants every single order in sales.orders numbered from largest to smallest net_total -- the complete global leaderboard.

TASK: Return all orders with order_id, order_date, net_total, and order_rank -- where order_rank is 1 for the largest order, 2 for the next, and so on. No grouping, no partitioning.

HINT: No PARTITION BY needed -- the whole table is one window. ROW_NUMBER gives unique numbers even when two orders have the same net_total.

Answer

✅ ANSWER

```
SELECT order_id, order_date, net_total,
       ROW_NUMBER() OVER (
           ORDER BY net_total DESC
       ) AS order_rank
FROM sales.orders;
```

Medium: Product Price Tiers with DENSE_RANK

MEDIUM

SCENARIO: The pricing analyst is building a tiered-discount strategy. Products at the same price share a tier. The next price level is the next tier -- no gaps. She tried RANK() but got 1, 1, 1, 4 when three products shared a price, which broke her discount table.

TASK: For every product, add a price_tier column using DENSE_RANK globally by price descending. Verify that consecutive distinct prices always produce consecutive tier numbers.

HINT: 'Same price = same tier' allows ties. 'No gaps in tier numbers' requires DENSE_RANK, not RANK. RANK would skip tier numbers after ties.

Answer

✅ ANSWER

```
SELECT p.product_name, dc.category_name, p.price,
       DENSE_RANK() OVER (
           ORDER BY p.price DESC
       ) AS price_tier
FROM products.products p
JOIN core.dim_brand db ON p.brand_id = db.brand_id
JOIN core.dim_category dc ON db.category_id = dc.category_id
ORDER BY price_tier;
```

Hard: Deterministic Per-Store Salary Ranking

HARD

SCENARIO: A QA bug was filed: the employee ranking page changes every refresh. Two employees both earn 45,000 at Store 12, and their order flips randomly. The root cause: ROW_NUMBER with ORDER BY salary DESC only -- no tiebreaker. You need deterministic ordering.

TASK: For each employee, add a salary_rank column using ROW_NUMBER partitioned by store_id, ordered by salary descending. Break ties by employee_id ascending so results are stable between runs. Compare ROW_NUMBER, RANK, and DENSE_RANK side by side.

HINT: Adding employee_id as a secondary ORDER BY column guarantees deterministic results. Without it, tied rows get random numbering on every run -- a production stability bug.

Answer

✅ ANSWER

```
SELECT first_name, store_id, salary,
       ROW_NUMBER() OVER (
           PARTITION BY store_id
           ORDER BY salary DESC,
                    employee_id ASC
       ) AS rn,
       RANK() OVER (
           PARTITION BY store_id
           ORDER BY salary DESC
       ) AS rnk,
       DENSE_RANK() OVER (
           PARTITION BY store_id
           ORDER BY salary DESC
       ) AS drnk
FROM stores.employees
ORDER BY store_id, rn;
```

Crazy: The Nth Highest Salary

CRAZY

SCENARIO: This is THE most-asked SQL interview question in history: 'Find the 3rd highest unique salary in the company and list every employee who earns it.' ROW_NUMBER returns only one row (wrong). RANK might skip tier 3 if ties exist above (wrong). Only DENSE_RANK on distinct values guarantees the correct answer.

TASK: Use a CTE with DENSE_RANK on DISTINCT salary values to find the 3rd-highest unique salary. Then join back to employees to show everyone earning that amount with their name and role.

HINT: DISTINCT inside the CTE is critical -- without it, the ranking runs on thousands of employee rows instead of unique salary values. Change tier = 3 to any N to generalize.

Answer

✅ ANSWER

```
WITH salary_tiers AS (
    SELECT DISTINCT salary,
           DENSE_RANK() OVER (
               ORDER BY salary DESC
           ) AS tier
    FROM stores.employees
)
SELECT e.first_name, e.last_name,
       e.role, e.salary
FROM stores.employees e
JOIN salary_tiers st
  ON e.salary = st.salary
WHERE st.tier = 3
ORDER BY e.first_name;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

ROW_NUMBER: always unique (1,2,3,4,5) -- use for sequencing, deduplication, pagination. RANK: ties share + skip (1,1,3,4,5) -- use for competitive leaderboards. DENSE_RANK: ties share, no skip (1,1,2,3,4) -- use for Nth highest salary. Rule of thumb: 'Nth highest UNIQUE value' is always DENSE_RANK. Always add a tiebreaker to ROW_NUMBER's ORDER BY for deterministic results.

PRO TIP

PRO TIP

In interviews, mention all three approaches and explain why DENSE_RANK wins for '2nd highest salary': ROW_NUMBER gives the 2nd person (might still have the highest salary if there is a tie). RANK might skip rank 2 entirely. DENSE_RANK guarantees the 2nd unique value. Interviewers love candidates who compare approaches.

COMMON MISTAKE

Common Mistakes with Ranking Functions

Mistake 1: Using ROW_NUMBER for competitive ranking

Two people score 100. ROW_NUMBER gives one rank 1 and the other rank 2. The second person is furious -- they should share 1st place.

Fix:

Use RANK() or DENSE_RANK() when ties should mean equal placement.

Mistake 2: Filtering ROW_NUMBER in WHERE directly

SELECT *, ROW_NUMBER() OVER(...) AS rn FROM employees WHERE rn <= 3 -- ERROR! Window functions run AFTER WHERE.

Fix:

Wrap in a CTE: WITH Ranked AS (SELECT *, ROW_NUMBER()...) SELECT * FROM Ranked WHERE rn <= 3.

STORY

Quartiles, Deciles, Percentiles -- One Function

The marketing team asks: 'Divide all customers into 4 equal groups by total spend. Label them Platinum, Gold, Silver, Bronze.' They want a quartile bucket, not a rank number.

ROW_NUMBER gives 1 through 50,000 -- unique but useless for grouping. RANK gives ties + gaps. DENSE_RANK gives tiers -- but how many tiers? Depends on the data.

NTILE(4) does exactly what marketing wants: splits the ordered rows into 4 roughly equal groups. Group 1 = top 25%, Group 2 = next 25%, and so on. Change 4 to 10 for deciles, 100 for percentiles.

This is the foundation of RFM segmentation on Cohort & RFM -- you will use NTILE to score customers on Recency, Frequency, and Monetary value.

DEFINITION

NTILE(N)

Technical:

NTILE(N) distributes ordered rows into N roughly equal groups (buckets), numbered 1 through N. If the row count is not evenly divisible by N, earlier buckets get one extra row. Requires ORDER BY inside OVER() to determine the sort order before bucketing.

Simple:

Think of NTILE as 'deal cards into N piles.' Sort all rows, then deal them round-robin into N groups. Group 1 is the top slice, Group N is the bottom slice. NTILE(4) = quartiles. NTILE(10) = deciles. NTILE(100) = percentiles.

Distribution rule:

100 rows / NTILE(4) = 25 per group (perfect). 103 rows / NTILE(4) = 26, 26, 26, 25. The first 3 groups absorb the remainder.

NTILE -- Syntax & Comparison

```sql
-- NTILE(4): Split into 4 equal groups (quartiles)
SELECT customer_id, total_spend,
       NTILE(4) OVER (ORDER BY total_spend DESC)
         AS spend_quartile
FROM customer_summary;

-- Group 1 = top 25% spenders (Platinum)
-- Group 2 = next 25% (Gold)
-- Group 3 = next 25% (Silver)
-- Group 4 = bottom 25% (Bronze)

-- NTILE(10): Deciles
-- NTILE(100): Percentiles
-- NTILE(5): Quintiles
```

Easy: Customer Spend Quartiles

EASY

SCENARIO: The loyalty team is rolling out a 4-tier rewards program: Platinum (top 25% spenders), Gold, Silver, Bronze. They need every customer tagged with their quartile based on total lifetime spend.

TASK: First aggregate total spend per customer in a CTE. Then apply NTILE(4) ordered by total spend descending. Return customer_id, total_spend, and spend_quartile (1 = Platinum through 4 = Bronze).

HINT: NTILE needs a pre-aggregated dataset. The CTE computes per-customer totals from sales.orders. NTILE(4) in the outer query splits those totals into 4 equal buckets.

Answer

✅ ANSWER

```
WITH customer_spend AS (
    SELECT cust_id,
           SUM(net_total) AS total_spend
    FROM sales.orders
    GROUP BY cust_id
)
SELECT cust_id, total_spend,
       NTILE(4) OVER (
           ORDER BY total_spend DESC
       ) AS spend_quartile
FROM customer_spend;
```

Medium: Product Price Deciles by Category

MEDIUM

SCENARIO: The pricing team wants to understand the price distribution WITHIN each category. They want products divided into 10 price deciles per category -- decile 1 = most expensive 10% in that category, decile 10 = cheapest 10%.

TASK: Use NTILE(10) with PARTITION BY category and ORDER BY price DESC. Return product_name, category, price, and price_decile.

HINT: PARTITION BY category + NTILE(10) creates separate decile rankings within each category. Electronics products are bucketed against other electronics, not against groceries.

Answer

✅ ANSWER

```
SELECT p.product_name, dc.category_name, p.price,
       NTILE(10) OVER (
           PARTITION BY dc.category_name
           ORDER BY p.price DESC
       ) AS price_decile
FROM products.products p
JOIN core.dim_brand db ON p.brand_id = db.brand_id
JOIN core.dim_category dc ON db.category_id = dc.category_id
ORDER BY dc.category_name, price_decile;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

NTILE(N) splits sorted rows into N equal groups. NTILE(4) = quartiles, NTILE(10) = deciles, NTILE(100) = percentiles. Combine with PARTITION BY to bucket within groups. NTILE is the foundation of RFM segmentation (Cohort & RFM) -- scoring customers by Recency, Frequency, and Monetary value into quartile buckets.

PRO TIP

PRO TIP

NTILE is 'rough' bucketing -- it splits by position, not by value. Two customers spending 99,999 and 100,001 could end up in different quartiles if the boundary falls between them. For exact value-based thresholds, use CASE WHEN. For statistical percentiles, use PERCENTILE_CONT (Median & Percentiles). NTILE is perfect for 'divide into equal-sized groups' use cases like RFM.

STORY

The #1 SQL Interview Question

Merchandising wants: 'Top 3 products by revenue in EACH category.' RetailMart has 10 categories and 1,500 products. The result should be about 30 rows (3 per category), not 3 total.

Why naive attempts fail: ORDER BY revenue DESC LIMIT 3 returns 3 rows across the whole table -- one category dominates. GROUP BY category collapses products away -- you lose product identity.

The solution is always TWO steps: Step 1 (CTE): compute per-product revenue, then rank within category using a window function. Step 2 (outer query): filter WHERE rank <= 3.

The CTE is NOT optional. Window functions execute AFTER WHERE in the SQL pipeline. You cannot write WHERE ROW_NUMBER() <= 3 directly -- the rank does not exist yet when WHERE runs. The CTE materializes the rank; the outer query filters on it.

DEFINITION

Top N per Category Pattern

Technical:

Wrap a ranking window function inside a CTE to generate per-group rankings, then filter in the outer query WHERE rank <= N. This two-step approach is mandatory because SQL evaluates WHERE before window functions.

Which ranking function to choose:

ROW_NUMBER: exactly N rows per group, no ties. Good for 'latest order per customer', 'one flagship product per brand'. DENSE_RANK: may return MORE than N rows if ties exist. Good for 'top 3 salary tiers per store'. RANK: same tie behavior as DENSE_RANK for top N but gap numbers look odd. Rarely used here.

The key question:

'Do I want exactly N rows, or N tiers with ties included?' Exactly N = ROW_NUMBER. N tiers = DENSE_RANK.

Top N per Category -- The Universal Template

```
-- STEP 1: Generate rankings inside a CTE
WITH ranked AS (
    SELECT category_col, detail_col, metric,
           ROW_NUMBER() OVER (
               PARTITION BY category_col
               ORDER BY metric DESC
           ) AS rn
    FROM source_table
)
-- STEP 2: Filter in outer query
SELECT * FROM ranked WHERE rn <= N;

-- VARIATION: Bottom N (worst performers)
-- Change DESC to ASC in ORDER BY!
```

Easy: Flagship Product per Brand

EASY

SCENARIO: RetailMart is redesigning the home page with a 'Brand Spotlight' carousel -- one hero product per brand, the most expensive. If two products tie, just pick one (marketing chooses the photo). Output: exactly one row per brand.

TASK: From products.products, return brand, product_name, price -- one row per brand -- showing the most expensive product. Use CTE + ROW_NUMBER + WHERE rn = 1.

HINT: 'Just pick one when tied' means ROW_NUMBER, not DENSE_RANK (which would return multiple rows for ties). PARTITION BY brand gives one window per brand.

Answer

✅ ANSWER

```
WITH ranked_products AS (
    SELECT db.brand_name, p.product_name, p.price,
           ROW_NUMBER() OVER (
               PARTITION BY db.brand_name
               ORDER BY p.price DESC
           ) AS rn
    FROM products.products p
    JOIN core.dim_brand db ON p.brand_id = db.brand_id
)
SELECT brand_name, product_name, price
FROM ranked_products
WHERE rn = 1;
```

Medium: Top 3 Salary Tiers per Store -- Ties Included

MEDIUM

SCENARIO: HR wants the top 3 salary TIERS within each store -- not the top 3 people, the top 3 pay levels. If 4 employees at Store 7 all earn 95,000, they ALL belong to tier 1 and must ALL appear. Using ROW_NUMBER would arbitrarily exclude some of them.

TASK: Use DENSE_RANK in a CTE, partitioned by store_id, ordered by salary descending. In the outer query, keep only tiers 1-3. Some stores may return more than 3 rows if ties exist.

HINT: 'Top N tiers + include ties' is always DENSE_RANK. ROW_NUMBER would drop ties. RANK would skip tier numbers after ties.

Answer

✅ ANSWER

```
WITH store_salaries AS (
    SELECT store_id, first_name, salary,
           DENSE_RANK() OVER (
               PARTITION BY store_id
               ORDER BY salary DESC
           ) AS salary_tier
    FROM stores.employees
)
SELECT store_id, first_name,
       salary, salary_tier
FROM store_salaries
WHERE salary_tier <= 3
ORDER BY store_id, salary_tier;
```

Hard: Most Recent Order per Customer

HARD

SCENARIO: The retention team is launching a win-back SMS campaign to customers inactive for 90+ days. They need each customer's single most recent order -- order_id, date, amount -- joined with name and email. The old correlated subquery with MAX(order_date) took 40 seconds on 150,000 orders.

TASK: CTE with ROW_NUMBER partitioned by cust_id, ordered by order_date descending. Outer query joins customers and filters WHERE rn = 1. Return first_name, email, order_id, order_date, net_total.

HINT: Window function scans orders ONCE. The old correlated subquery scanned once per customer (50,000 sub-scans). This is the standard 'one row per entity' pattern.

Answer

✅ ANSWER

```
WITH latest_orders AS (
    SELECT cust_id, order_id,
           order_date, net_total,
           ROW_NUMBER() OVER (
               PARTITION BY cust_id
               ORDER BY order_date DESC
           ) AS rn
    FROM sales.orders
)
SELECT c.first_name, c.email,
       lo.order_id, lo.order_date,
       lo.net_total
FROM latest_orders lo
JOIN customers.customers c
  ON lo.cust_id = c.customer_id
WHERE lo.rn = 1;
```

Crazy: Top 2 Orders per Customer per Status

CRAZY

SCENARIO: The ops team is triaging customer complaints. For each customer, they want the 2 biggest orders in EACH status bucket -- top 2 Delivered, top 2 Processing, top 2 Cancelled. This needs multi-column PARTITION BY creating micro-windows like (Customer 1001, Delivered), (Customer 1001, Cancelled).

TASK: CTE with ROW_NUMBER over PARTITION BY cust_id, status, ordered by net_total descending. Outer query keeps WHERE rn <= 2. Sort by cust_id, status, rn.

HINT: Multiple columns in PARTITION BY create combinatorial micro-windows. 50,000 customers times ~5 statuses = 250,000 micro-windows. PostgreSQL still does this in a single sorted pass.

Answer

✅ ANSWER

```
WITH status_ranked AS (
    SELECT o.cust_id, o.order_id,
           o.order_status, o.net_total,
           o.order_date,
           ROW_NUMBER() OVER (
               PARTITION BY o.cust_id,
                            o.order_status
               ORDER BY o.net_total DESC
           ) AS rn
    FROM sales.orders o
)
SELECT cust_id, order_id, order_status,
       net_total, order_date, rn
FROM status_ranked
WHERE rn <= 2
ORDER BY cust_id, order_status, rn;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

The Top N per Category pattern: (1) CTE generates rankings with a window function, (2) outer query filters WHERE rank <= N. This is mandatory -- you cannot filter window functions in WHERE directly. Choose ROW_NUMBER for exactly N rows, DENSE_RANK for N tiers with ties. Multi-column PARTITION BY creates nested groupings. This pattern replaces slow correlated subqueries.

PRO TIP

PRO TIP

In interviews, always clarify: 'Do you want exactly N rows per group, or N tiers with ties included?' This shows you understand the ROW_NUMBER vs DENSE_RANK distinction. Most interviewers are impressed when candidates ask this clarifying question before writing code.

COMMON MISTAKE

Common Mistakes with Top N per Category

Mistake 1: Using LIMIT instead of Top N pattern

ORDER BY salary DESC LIMIT 3 gives top 3 for the ENTIRE table, not per group. You would need a separate query for each group.

Fix:

PARTITION BY group_col in the window function handles ALL groups simultaneously in one query.

Mistake 2: Wrong ranking function for the requirement

Using ROW_NUMBER when ties should be included. If 3 people tie for 1st, ROW_NUMBER only keeps 1 of them.

Fix:

Use DENSE_RANK when 'top N' means 'top N tiers/levels' and all tied rows should appear.

INTERVIEW

Window Functions vs GROUP BY

Q: What is the difference between GROUP BY and window functions?

A: GROUP BY collapses multiple rows into one summary row per group -- individual rows are lost. Window functions compute aggregates across a group of rows but retain every individual row. With GROUP BY, you choose between detail and summary. With window functions, you get both on the same row. Window functions use the OVER() clause and execute after WHERE/GROUP BY/HAVING in the SQL pipeline.

INTERVIEW

ROW_NUMBER vs RANK vs DENSE_RANK

Q: When would you use each ranking function?

A: ROW_NUMBER for unique sequencing (deduplication, pagination, finding firsts) -- never ties. RANK for competitive leaderboards where ties share a position and the next rank skips (like Olympic medals). DENSE_RANK for 'Nth highest unique value' problems -- ties share but no gaps. For '2nd highest salary': ROW_NUMBER is wrong (gives 2nd person, not 2nd salary), RANK might have no rank 2 if ties exist above, DENSE_RANK always finds the 2nd unique value.

INTERVIEW

Top N per Group Pattern

Q: Find the top 5 selling products in every region.

A: This is a Top N per Category problem. Step 1: CTE that aggregates total revenue per product per region (JOIN orders, order_items, stores). Step 2: Add DENSE_RANK() OVER (PARTITION BY region ORDER BY revenue DESC) inside the CTE. Step 3: Outer query filters WHERE rank <= 5. I use DENSE_RANK because if two products tie in revenue, they should both be considered top 5.

INTERVIEW

Deduplication with ROW_NUMBER

Q: How do you identify and delete duplicate records?

A: CTE with ROW_NUMBER() OVER (PARTITION BY duplicate_columns ORDER BY id ASC) AS rn. The PARTITION BY defines what 'duplicate' means. ORDER BY decides which copy to keep (smallest ID = original). Every duplicate gets rn > 1. Filter WHERE rn = 1 to see originals. DELETE WHERE id IN (SELECT id FROM cte WHERE rn > 1) to remove duplicates.

REAL WORLD

REAL WORLD (1/2)

What You Achieved Today

1. OVER Clause & PARTITION BY

Window functions compute aggregates without collapsing rows. OVER() turns any aggregate into a window function. PARTITION BY creates per-group windows. You can use multiple window functions with different partitions in the same SELECT. They execute after WHERE in the SQL pipeline.

2. Ranking Functions

ROW_NUMBER: unique sequential numbers (1,2,3,4,5). RANK: ties + gaps (1,1,3,4,5). DENSE_RANK: ties, no gaps (1,1,2,3,4). Rule: 'Nth highest unique value' = DENSE_RANK. Always add a tiebreaker to ROW_NUMBER for deterministic results.

REAL WORLD

REAL WORLD (2/2)

3. NTILE -- Bucketing

NTILE(N) splits sorted rows into N equal groups. NTILE(4) = quartiles, NTILE(10) = deciles. Foundation for RFM segmentation on Cohort & RFM -- scoring customers into Recency, Frequency, and Monetary quartile buckets.

4. Top N per Category

The most-asked SQL interview pattern. CTE generates rankings, outer query filters WHERE rank <= N. Choose ROW_NUMBER for exactly N rows, DENSE_RANK for N tiers with ties. Multi-column PARTITION BY creates nested groupings.

Where This Is Used:

- Amazon/Flipkart: 'Top 3 products per category' -- literally the same pattern
- Zomato/Swiggy: 'Top 3 restaurants per area by rating' -- runs thousands of times per minute
- Banks: Running totals, competitive rankings, credit scoring leaderboards
- HR: Pay equity reviews, salary band analysis, first-hire reports
- Data Engineering: Deduplication using ROW_NUMBER = 1 -- every ETL pipeline

Next up (Window Functions Part 2): Window aggregates (SUM, AVG with frames) and running totals. Adding ORDER BY inside OVER turns a total into a running total -- one syntax change, dozens of analyses.

Window Functions Part 1 -- Ranking & Bucketing

-- OVER() with Aggregates

```
-- Per-store average alongside each row
SELECT salary,
  AVG(salary) OVER(PARTITION BY store_id)
    AS store_avg
FROM stores.employees;
```

-- OVER() without PARTITION BY

```
-- Compare each row to company average
SELECT salary,
  AVG(salary) OVER() AS company_avg,
  salary - AVG(salary) OVER() AS diff
FROM stores.employees;
```

-- ROW_NUMBER()

```
-- Unique sequence, no ties
SELECT ROW_NUMBER() OVER(
  PARTITION BY store_id
  ORDER BY salary DESC, employee_id
) AS rn
-- Always add tiebreaker column!
```

Window Functions Part 1 -- Ranking & Bucketing

```
-- RANK() & DENSE_RANK()

RANK():       1, 1, 3, 4 (gaps after ties)
DENSE_RANK(): 1, 1, 2, 3 (no gaps)
-- Nth highest: DENSE_RANK always
-- Leaderboard: RANK for positions


-- Top N per Category

WITH Ranked AS (
  SELECT *, DENSE_RANK() OVER(
    PARTITION BY category
    ORDER BY revenue DESC) AS rn
  FROM data
)
SELECT * FROM Ranked WHERE rn <= 3;


-- Deduplication Pattern

WITH Dupes AS (
  SELECT *, ROW_NUMBER() OVER(
    PARTITION BY email
    ORDER BY id ASC) AS rn
  FROM customers.customers
)
SELECT * FROM Dupes WHERE rn = 1;
```

Window Functions Part 1 -- Ranking & Bucketing

-- NTILE(N) -- Bucketing

```
-- Quartiles: NTILE(4)
SELECT cust_id, total_spend,
  NTILE(4) OVER(ORDER BY total_spend DESC)
    AS quartile
FROM customer_summary;
-- 1=top 25%, 4=bottom 25%
-- NTILE(10)=deciles, NTILE(100)=percentiles
```

-- Nth Highest Value

```
WITH Tiers AS (
  SELECT DISTINCT salary,
    DENSE_RANK() OVER(ORDER BY salary DESC)
      AS tier
  FROM stores.employees
)
SELECT salary FROM Tiers WHERE tier = 3;
```

CHALLENGE

CHALLENGE (1/6)

17 Practice Problems

OVER Clause & PARTITION BY (1-5)

CHALLENGE

CHALLENGE (2/6)

1.: Show every employee's salary PLUS their store's average PLUS the company average -- three columns, one SELECT, no GROUP BY.

2.: For each product, show product_name, price, and the average price for that product's category using OVER(PARTITION BY category).

3.: For each order, show order_id, net_total, and what percentage that order represents of its store's total revenue.

4.: Show each store's total order count alongside the company-wide total order count on the same row (two different OVER clauses).

5.: For each employee, show their salary, the MAX salary in their store, and the MIN salary -- all without GROUP BY.

CHALLENGE

CHALLENGE (3/6)

Ranking Functions (6-10)

CHALLENGE

CHALLENGE (4/6)

6.: Number each customer's orders chronologically (1st, 2nd, 3rd) using ROW_NUMBER partitioned by cust_id.

7.: Rank all products by price using all three functions side by side. Find a row where RANK and DENSE_RANK produce different numbers.

8.: Find the 5th highest unique salary in stores.employees using DENSE_RANK. List every employee who earns that salary.

9.: Use ROW_NUMBER to rank employees by joining_date ASC within each store. Show only the first hire per store.

10.: Write a deduplication query: identify duplicate (first_name, store_id) combinations and keep only the lowest employee_id per group.

NTILE -- Bucketing (11-12)

CHALLENGE

CHALLENGE (5/6)

11.: Divide all customers into 4 spend quartiles using NTILE(4). Show customer_id, total_spend, and quartile. How many customers are in each quartile?

12.: Within each product category, divide products into 5 price quintiles using NTILE(5) with PARTITION BY category. Which quintile has the widest price range?

Top N per Category (13-17)

CHALLENGE

CHALLENGE (6/6)

13.: Find the most expensive product in each category using CTE + ROW_NUMBER.

14.: Find the top 3 employees by salary in each store using DENSE_RANK (include ties).

15.: Find each customer's most recent order -- return customer name, order_id, order_date, net_total.

16.: Find the bottom 3 products by price in each category (flip DESC to ASC).

17.: (Crazy) For each customer, find their 2 highest-value orders per order status (multi-column PARTITION BY cust_id, status).

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Basic Queries
- ✓ Data Types
- ✓ Constraints
- ✓ Filtering
- ✓ Scalar Functions
- ✓ Conditional Logic
- ✓ Aggregates
- ✓ Joins Part 1
- ✓ Joins Part 2
- ✓ Self, Cross & Sets
- ✓ Subqueries Part 1
- ✓ CTEs & Correlated
- ✓ DB Concepts
- ✓ Review & Practice
- 1 Window Functions P1 ← HERE